NAVIGATOR

선설계 후개발 구조를 통한 **차세대 AI 코딩 패러다임**의 완성

CONTENTS

PROBLEM

01 시장분석 03p
02 기존 서비스 분석 04p
03 문제점 제시 06p

SOLUTION

01 UI/UX 08p
02 시스템 아키텍쳐 09p
03 핵심기능 10p
04 시연영상 13p

CONCLUSION

01 기대효과 14p
02 로드맵 15p
03 Q&A 16p

시장분석

01

AI 도구 시장, 골든 타임 진입

- 2022년 1.83Bn(18.3억), 2032년 20.7Bn(207억) 예측
- 약 **CAGR 22.2%** 고속 성장 기대
- AI 코딩 도구 시장의 **폭발적 확대**

02

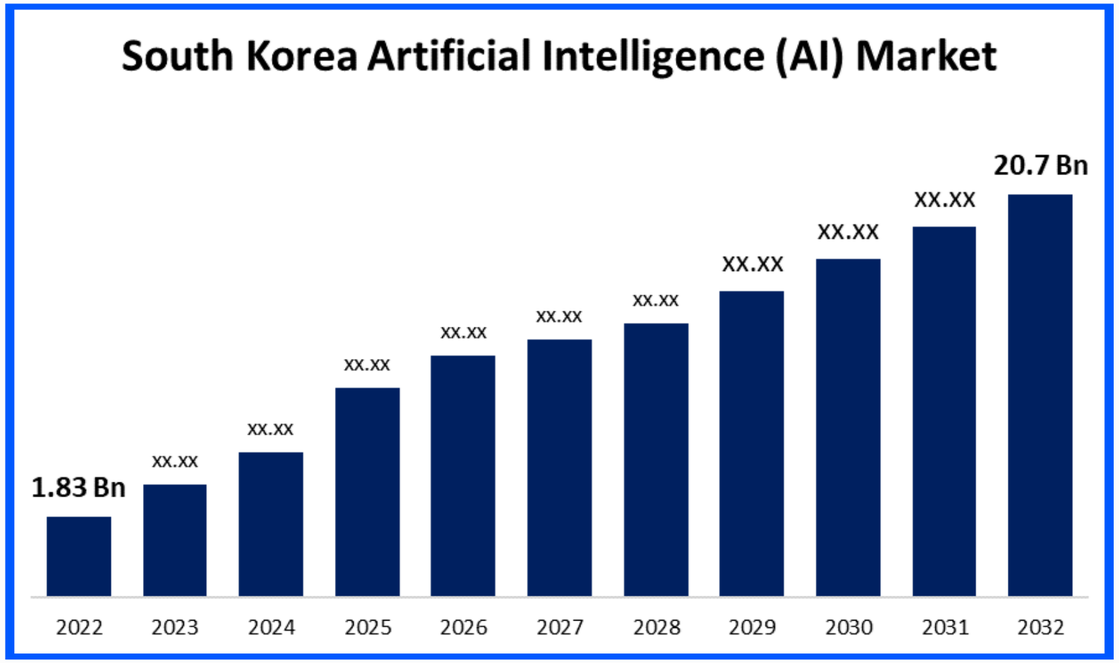
작성과정의 마찰과 비효율

- 초기 세팅 및 보일러플레이트 코드 작성에 전체 개발 시간의 약 40% 이상 소모
- 개발자가 비즈니스 로직(핵심 가치) 설계보다 단순 구현에 리소스를 빼앗기는 구조

03

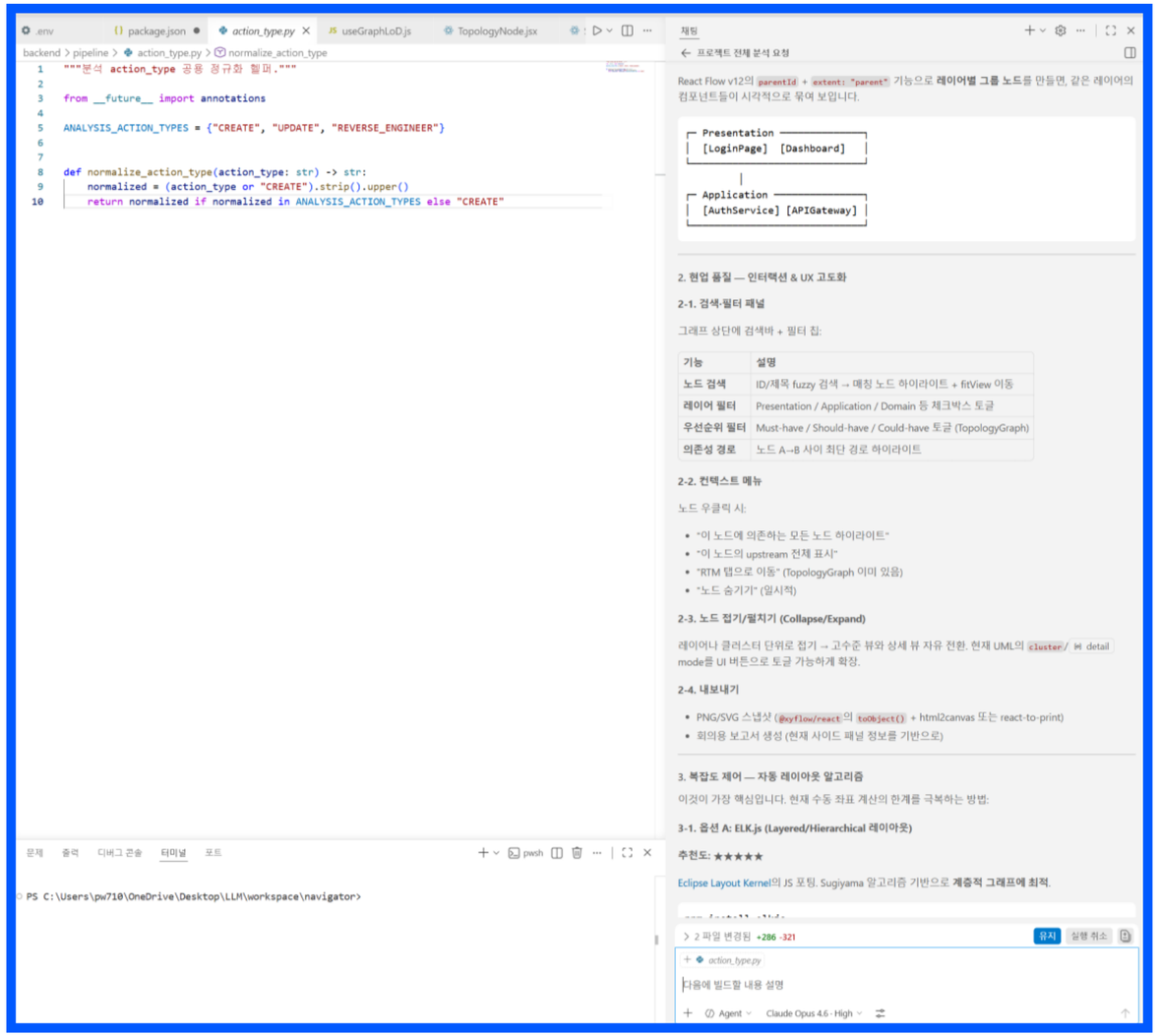
이해와 컨텍스트 단절

- 기존 코드를 읽고 파악하는 데 많은 시간 소요
- 수만 줄에 달하는 시스템을 파악하고 유지보수하기 어려운 한계 직면



TBRC, MarketsandMarkets, Future Market Insights, 국내 AI·디지털 시장 전망 보고서

기존 서비스 분석(COPILOT)



핵심기능

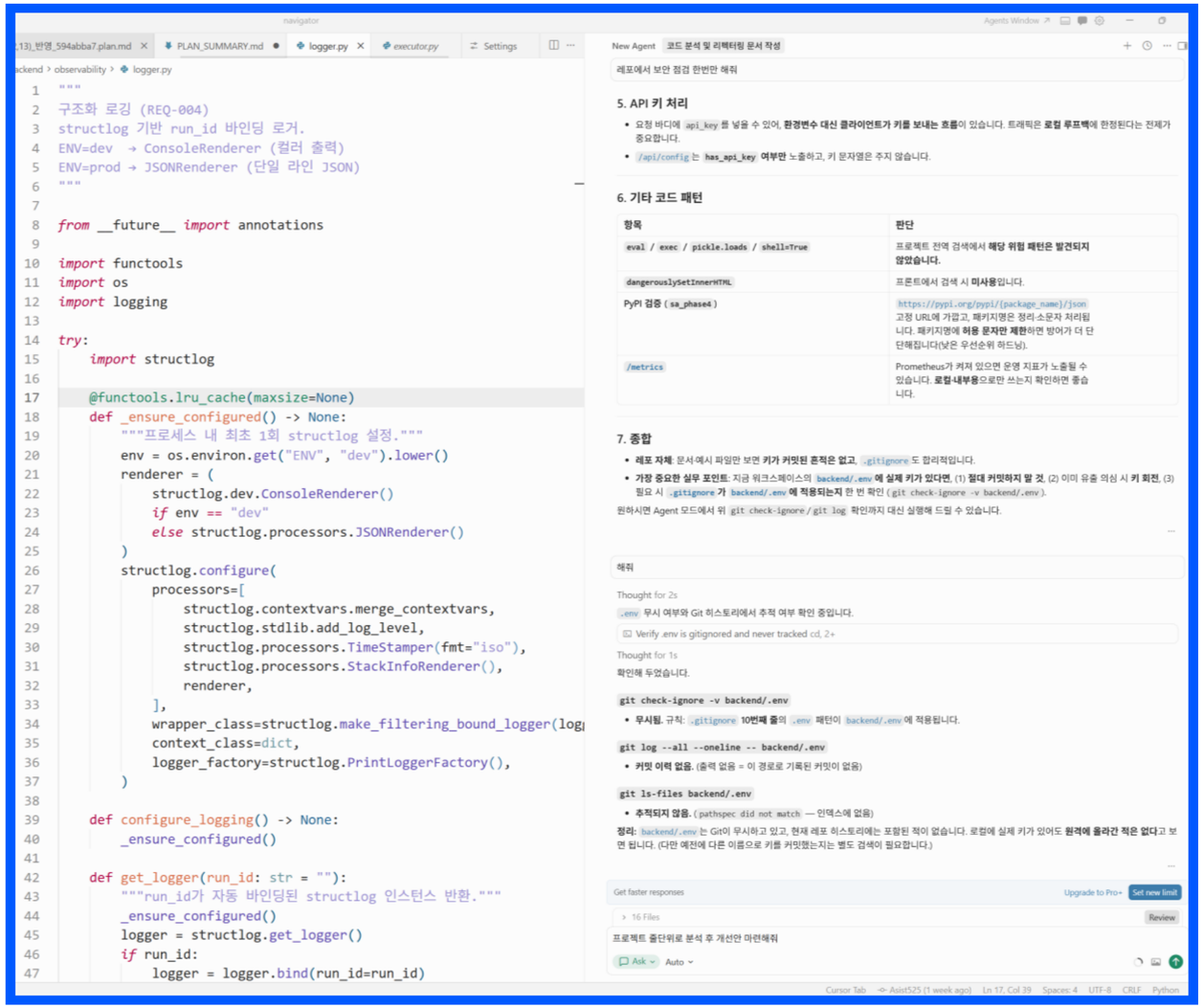
- **멀티스텝 편집 및 파일 자율 제어**: 개발자의 개입을 최소화하고 여러 파일에 걸친 코드 생성 및 수정 수행

- **런타임 에러 자가 치유** (Self-Heal): 에러 발생 시 AI가 스스로 디버깅 플로우를 가동하여 수정 코드 제안

COPILOT의 한계점

- **컨텍스트 파편화**: 대규모 레포(1,000+ 파일) 원거리 의존성 파악 한계
- **엔지니어링 '설계' 부재**: PRD, 아키텍처 등 구조적 산출물 생성 불가
- **영향도 검증 불가**: 변경된 코드가 전체 시스템에 미치는 파급 효과 추적 불가

기존 서비스 분석(CURSOR)



핵심기능

Agent Mode: 프로젝트 전체 파일 대상 자율 편집, 파일 생성/이동/삭제, 쉘 명령 실행, 테스트 통과까지 반복

Composer: 프로젝트 범위 대규모 코드 Diff 생성, 광범위 리팩토링, 전체 레포 컨텍스트 유지

Background Agent: 에디터가 닫혀 있어도 비동기 작업 수행, 최대 8개 병렬 (git worktree 기반 격리)

Subagent: 독립 컨텍스트 윈도우를 가진 하위 에이전트, 코드베이스 탐색/쉘 명령/브라우저 자동화 전문

CURSOR의 한계점

토큰 윈도우의 한계: 에이전트/서브에이전트당 ~100K 토큰. 태스크가 윈도우를 초과하면 이전 변경 사항을 "잊어버림"

서브에이전트 무상태: 서브에이전트는 매 할당마다 새로 시작

장기 품질 저하 실증: Cursor 도입 후 정적 분석 경고 30% 및코드 복잡도 41% 증가가 보고됨

추가 문제점 제시

01

맹목적 코드 수용

개발자가 꼼꼼한 리뷰 없이 겉보기에만 작동하는 코드를 무비판적으로 통과시켜, 보이지 않는 **구조적 붕괴와 잠재적 버그**를 프로덕션에 그대로 방치합니다.

Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review

02

생산성의 역설

도입 초기에는 빨라 보이나, 무분별한 코딩으로 **정적 분석 경고가 쌓이고 코드 복잡도가 폭증**하여 결국 장기적인 개발 속도를 하락시킵니다.

Does AI-Assisted Coding Deliver? A Difference-in-Differences Study of Cursor's Impact on Software Projects

03

치명적 패키지 환각

LLM이 약 19.7%의 확률로 존재하지 않는 가짜 라이브러리를 제안하며, 이를 수락할 경우 해커의 **악성 패키지**가 설치되는 심각한 보안 틈새가 발생합니다.

We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs

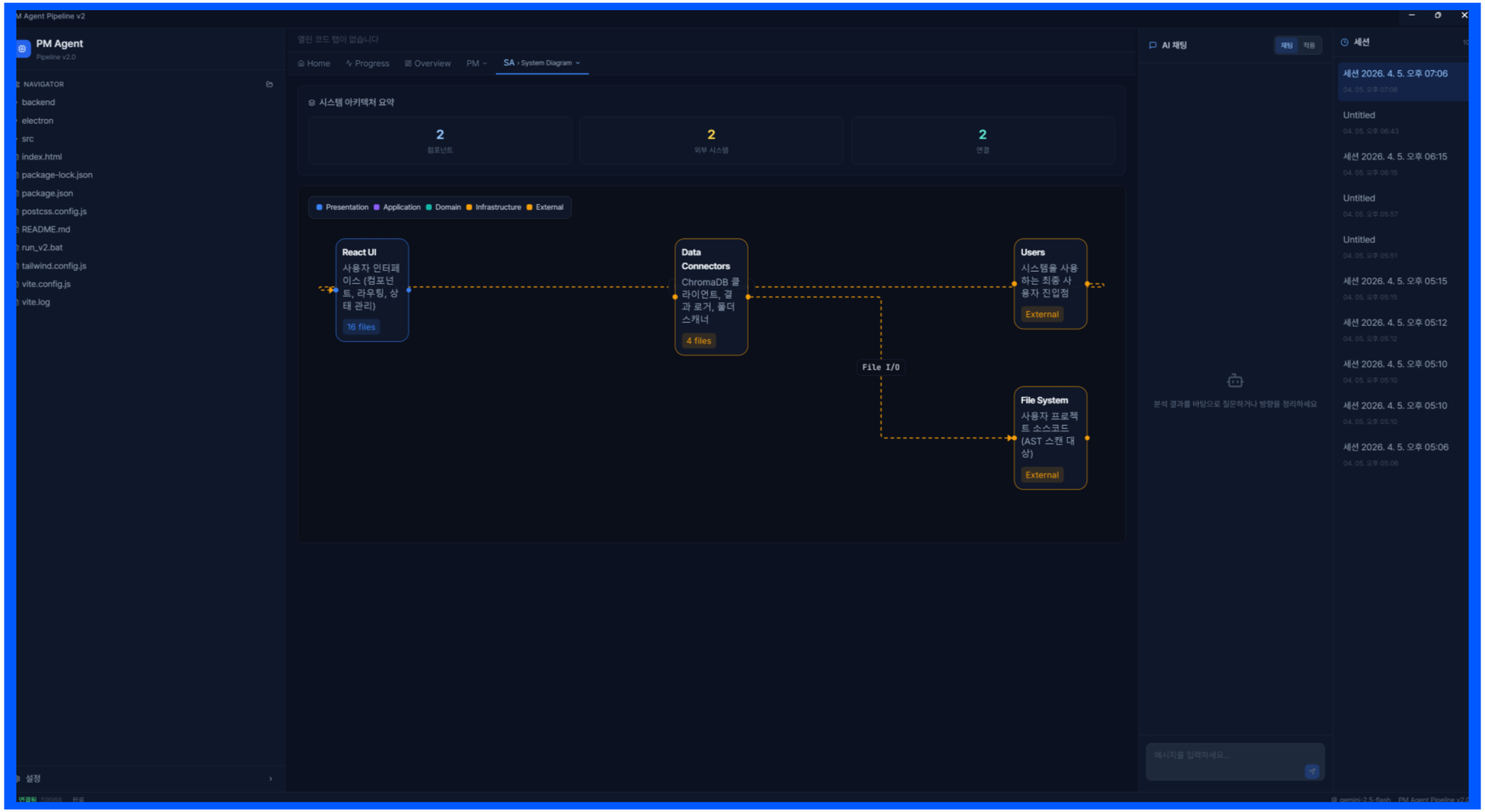
PART 2. 그래서 NAVIGATOR는 무엇이 다른가?

요구사항 정의

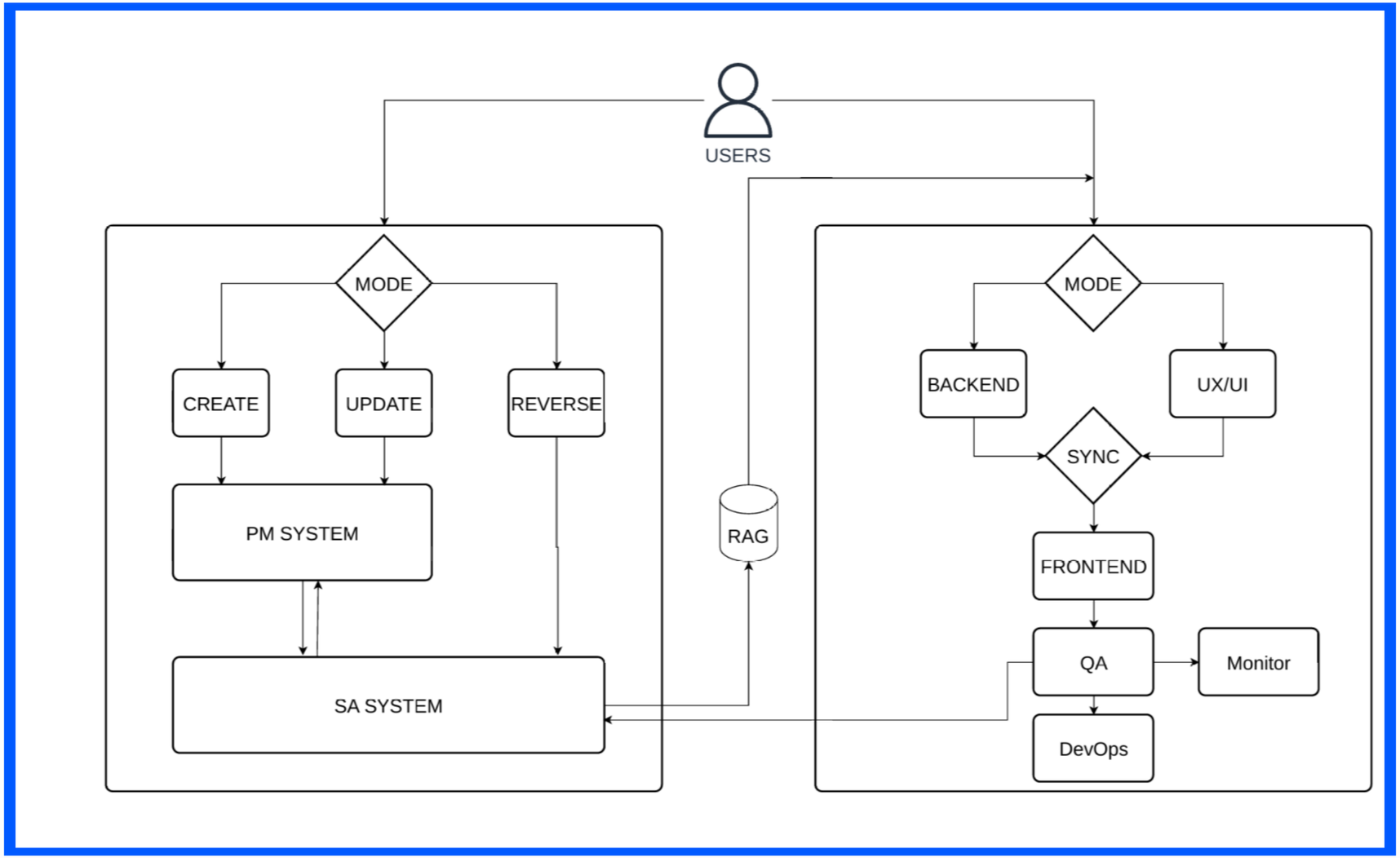
아키텍처 매핑

의존성 샌드박스 사전 검증

UI/UX


M Agent Pipeline v2
PM Agent
Pipeline v2.0
NAVIGATOR
backend
electron
src
index.html
package-lock.json
package.json
postcss.config.js
README.md
run_v2.bat
tailwind.config.js
vite.config.js
vite.log
설정
열린 코드 탭이 없습니다
Home
Progress
Overview
PM
SA › System Diagram
시스템 아키텍처 요약
2
컴포넌트
2
외부 시스템
2
연결
Presentation
Application
Domain
Infrastructure
External
React UI
사용자 인터페이스 (컴포넌트, 라우팅, 상태 관리)
16 files
Data Connectors
ChromaDB 클라이언트, 결과 로거, 폴더 스캐너
4 files
Users
시스템을 사용하는 최종 사용자 진입점
External
File I/O
File System
사용자 프로젝트 소스코드 (AST 스캔 대상)
External
AI 채팅
채팅
적용
분석 결과를 바탕으로 질문하거나 방향을 정리하세요
메시지를 입력하세요...
세션
세션 2026. 4. 5. 오후 07:06
Untitled
세션 2026. 4. 5. 오후 06:15
Untitled
Untitled
세션 2026. 4. 5. 오후 05:15
세션 2026. 4. 5. 오후 05:12
세션 2026. 4. 5. 오후 05:10
세션 2026. 4. 5. 오후 05:10
세션 2026. 4. 5. 오후 05:06

시스템 아키텍쳐


USERS
MODE
CREATE
UPDATE
REVERSE
PM SYSTEM
SA SYSTEM
RAG
MODE
BACKEND
UX/UI
SYNC
FRONTEND
QA
Monitor
DevOps

핵심기능: 요구사항 정의

ID	카테고리	설명	우선순위	의존성	테스트 기준
REQ-001	Frontend	게시물 목록을 조회하는 UI를 제공한다.	Must-have	REQ-002	게시물 목록 화면에서 최소 1개 이...
REQ-002	Backend	게시물 목록을 조회하는 API를 제공한다.	Must-have	REQ-007	게시물 목록 조회 API 호출 시 유효...
REQ-003	Frontend	특정 게시물의 상세 내용을 조회하는 UI를 제공한다.	Must-have	REQ-004	특정 게시물 ID로 상세 화면 접속 시...

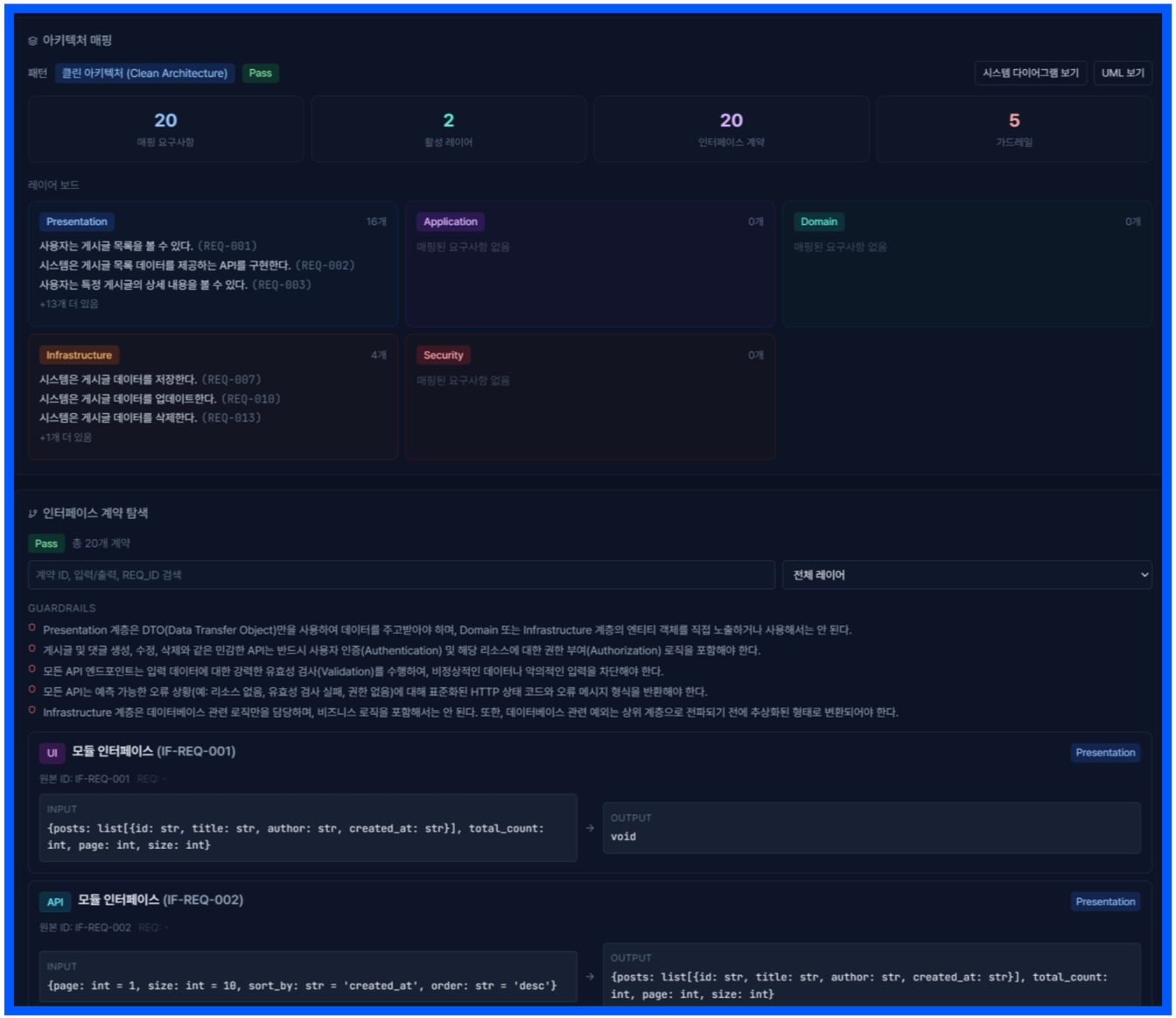
핵심기능: 무분별한 코딩을 막는 '단일 진실 공급원(SSOT)' 확립

[카테고리] 역할의 명확화: FRONTEND, BACKEND, SECURITY 등 에이전트의 작업 영역을 엄격히 분리하여 책임 소재 명확화

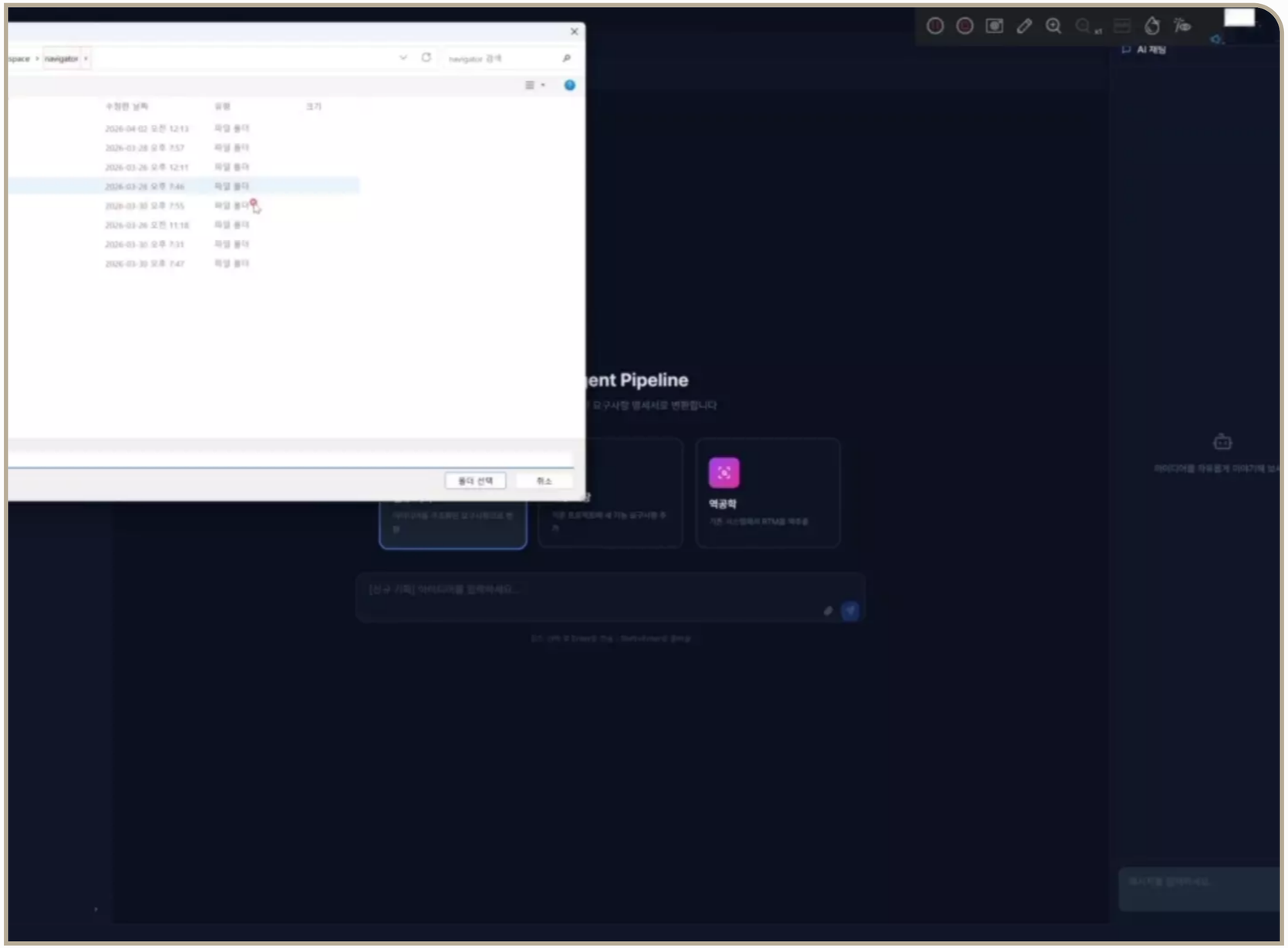
[우선순위] 전략적 자원 배분: MoSCoW 방법론(MUST/SHOULD/COULD HAVE)을 적용하여 핵심 비즈니스 로직부터 안전하게 구축

[의존성 & 테스트 기준] 결함 원천 차단: 기능 간의 사전 연결(의존성)과 통과 기준을 강제하여, 겉보기에만 도는 'Run-and-See' 맹점 및 환각(Hallucination) 방어

핵심기능: 아키텍처 매핑


아키텍처 매핑
패턴
클린 아키텍처 (Clean Architecture)
Pass
시스템 다이어그램 보기
UML 보기
20
매핑 요구사항
2
활성 레이어
20
인터페이스 계약
5
가드레일
레이어 보드
Presentation
16개
사용자는 게시글 목록을 볼 수 있다. (REQ-001)
시스템은 게시글 목록 데이터를 제공하는 API를 구현한다. (REQ-002)
사용자는 특정 게시글의 상세 내용을 볼 수 있다. (REQ-003)
+13개 더 있음
Application
0개
매핑된 요구사항 없음
Domain
0개
매핑된 요구사항 없음
Infrastructure
4개
시스템은 게시글 데이터를 저장한다. (REQ-007)
시스템은 게시글 데이터를 업데이트한다. (REQ-010)
시스템은 게시글 데이터를 삭제한다. (REQ-013)
+1개 더 있음
Security
0개
매핑된 요구사항 없음
인터페이스 계약 탐색
Pass
총 20개 계약
계약 ID, 입력/출력, REQ_ID 검색
전체 레이어
GUARDRAILS
Presentation 계층은 DTO(Data Transfer Object)만을 사용하여 데이터를 주고받아야 하며, Domain 또는 Infrastructure 계층의 엔티티 객체를 직접 노출하거나 사용해서는 안 된다.
게시글 및 댓글 생성, 수정, 삭제와 같은 민감한 API는 반드시 사용자 인증(Authentication) 및 해당 리소스에 대한 권한 부여(Authorization) 로직을 포함해야 한다.
모든 API 엔드포인트는 입력 데이터에 대한 강력한 유효성 검사(Validation)를 수행하여, 비정상적인 데이터나 악의적인 입력을 차단해야 한다.
모든 API는 예측 가능한 오류 상황(예: 리소스 없음, 유효성 검사 실패, 권한 없음)에 대해 표준화된 HTTP 상태 코드와 오류 메시지 형식을 반환해야 한다.
Infrastructure 계층은 데이터베이스 관련 로직만을 담당하며, 비즈니스 로직을 포함해서는 안 된다. 또한, 데이터베이스 관련 예외는 상위 계층으로 전파되기 전에 추상화된 형태로 변환되어야 한다.
UI
모듈 인터페이스 (IF-REQ-001)
Presentation
원본 ID: IF-REQ-001 REQ: -
INPUT
{posts: list[{id: str, title: str, author: str, created_at: str}], total_count: int, page: int, size: int}
OUTPUT
void
API
모듈 인터페이스 (IF-REQ-002)
Presentation
원본 ID: IF-REQ-002 REQ: -
INPUT
{page: int = 1, size: int = 10, sort_by: str = 'created_at', order: str = 'desc'}
OUTPUT
{posts: list[{id: str, title: str, author: str, created_at: str}], total_count: int, page: int, size: int}

시연영상


navigator
수정한 날짜
유형
크기
폴더 선택
취소
ent Pipeline
요구사항 명세서로 변환합니다
역공학
AI 채팅

기대효과

- 맹목적인 Run-and-See 관행 및 단순 코드 짜집기
- 통제 불가능한 에이전트로 인한 보이지 않는 구조적 결함 누적

TO BE

- **선설계 후개발**의 구조적 가드레일로 **견고한 아키텍처 확립**
- 버그 발생을 원천 차단하여 예측 가능한 고품질 엔지니어링 보장

- 에이전트의 단기 기억 상실 및 문맥 파편화 현상
- 무분별한 코딩에 따른 기술 부채 폭증 및 장기 개발 속도 하락

TO BE

- 검증된 디자인 패턴 기반의 뼈대 설계
- **코드 복잡도 통제**

- LLM의 환각으로 인한 가짜 패키지 제안 수용
- 악성 라이브러리 유입으로 인한 치명적인 시스템 보안 붕괴

TO BE

- **영향도를 사전 파악**하고 사이드 이펙트 방지
- 구축된 아키텍처와 요구사항을 직관적인 시각 자료로 **자동 산출**

로드맵

SA / PM 시스템 고도화

에이전트 축소/확대, 시스템 아키텍쳐 고도화, UI/UX 고도화, 파이썬, js이외의 코드 분석구조 구현, 리펙터링

AI 개발 시스템 구현

BACKEND, UI/UX, FRONTEND 시스템 구현

QA / 모니터링 시스템 구현

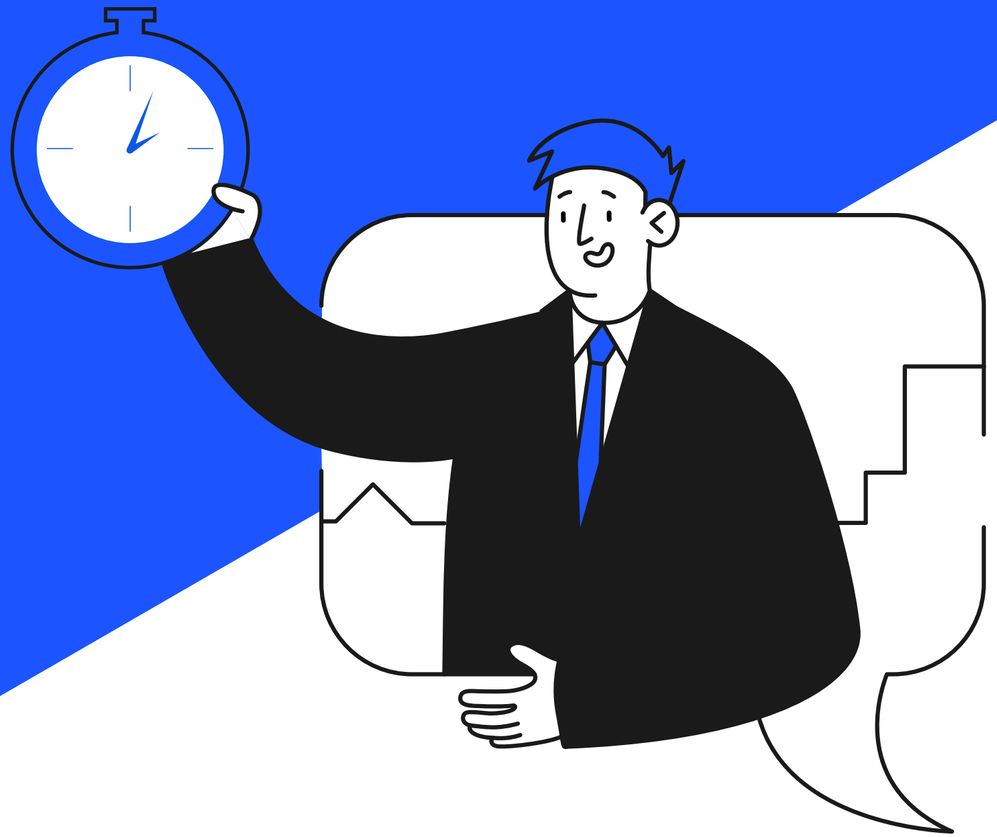
QA 및 모니터링 시스템 구현, RAG 모니터링 시스템 구현

협업기능 구현

다중접속구조 구현 및 고도화, github, slack 등 협업도구 연동

Q&A

'선설계 후개발' NAVIGATOR였습니다. 질문 있으시면 편하게 말씀해 주세요."



핵심기능: 시스템 다이어그램